

Segmentation

1 Core Idea

Definition

Segmentation is a memory-management scheme in which a process has multiple independent logical address spaces called **segments**.

With pure paging, the virtual address space is one long sequence of addresses:

0, 1, 2, ..., maximum address

This is called a **one-dimensional address space**. Segmentation changes the model. Instead of one long address space, a program can have separate address spaces for different logical parts of the program.

Segmented Address

$$\text{address} = (\text{segment number}, \text{offset inside that segment})$$

2 Why One Address Space Is Limiting

A compiler may build several large tables while compiling a program:

- source text;
- symbol table;
- constants table;
- parse tree;
- call stack.

In a one-dimensional virtual address space, all these structures must be placed in one linear layout.

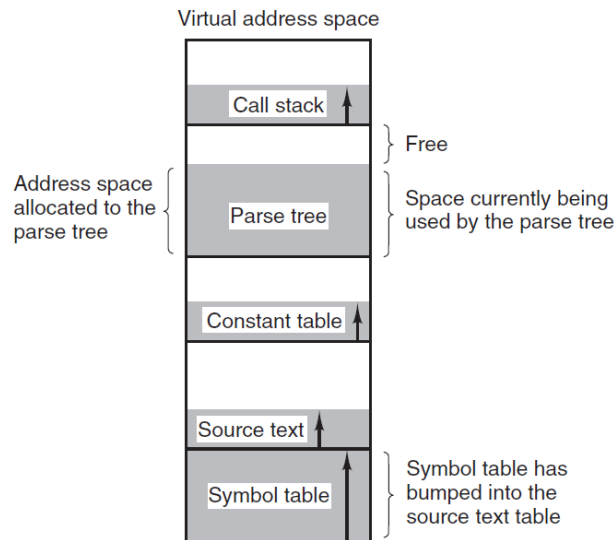


Figure 3-30. In a one-dimensional address space with growing tables, one table may bump into another.

Problem in the Figure

The symbol table has grown until it bumps into the source-text area. Even if other parts of the address space still have free space, this specific region is full.

3 The Need for Independent Growth

Different compiler tables grow differently:

Structure	Growth behavior
Source text	Usually grows as input is saved.
Symbol table	Grows as new names and attributes are discovered.
Constants table	Grows as constants appear in the program.
Parse tree	Grows as syntax is analyzed.
Call stack	Grows and shrinks during procedure calls.

The problem is not that virtual memory is too small in total. The problem is that each table is trapped inside a prechosen region of one linear address space.

4 Segmented Memory

Segmentation gives each logical structure its own segment.

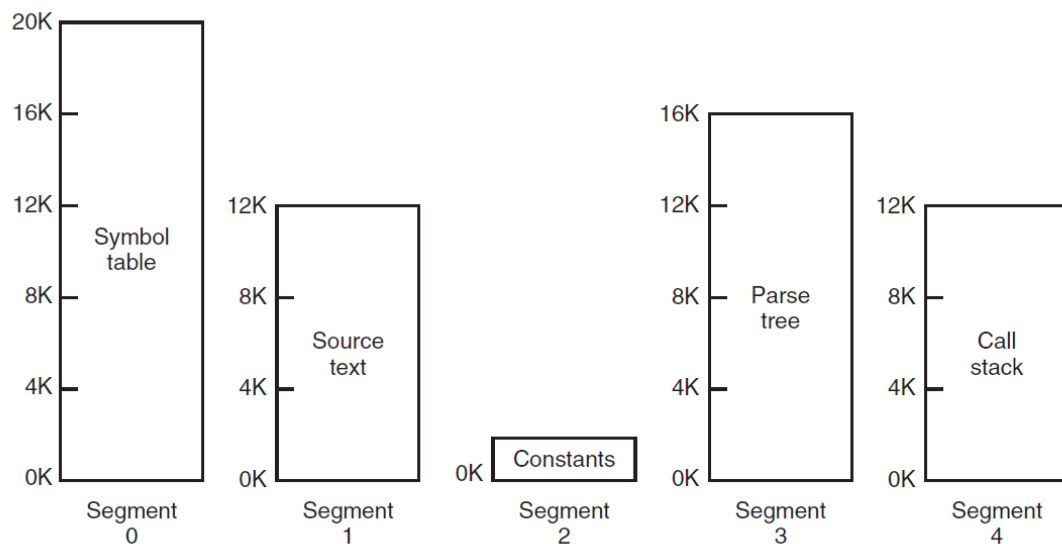


Figure 3-31. A segmented memory allows each table to grow or shrink independently of the other tables.

Figure Meaning

Each compiler table is placed in a separate segment. The symbol table, source text, constants, parse tree, and call stack can grow or shrink independently.

Each segment is a separate linear address space starting at address 0:

Segment 0 : $0 \rightarrow \text{limit}_0$

Segment 1 : $0 \rightarrow \text{limit}_1$

Segment 2 : $0 \rightarrow \text{limit}_2$

The limits may be different because segments do not have to be the same size.

5 Logical Meaning of a Segment

A segment is not just a fixed-size block. It is a logical unit that the programmer or compiler can understand.

Segment may contain	Example
Procedure	Code for one function or module.
Array	A large table or matrix.
Stack	Procedure-call frames and local variables.
Data structure	Symbol table, parse tree, or constants table.

Key Difference

Paging divides memory into equal-size physical units. Segmentation divides a program into meaningful logical units.

6 Address Translation Idea

In segmentation, an address has two parts:

$$(s, d)$$

where:

- s is the segment number;
- d is the offset inside that segment.

The hardware checks whether d is inside the segment limit. If it is valid, the physical address can be formed using the segment base.

Basic Segmentation Translation

$$\text{physical address} = \text{base}[s] + d$$

$$d < \text{limit}[s]$$

If d is greater than or equal to the segment limit, the access is invalid and the OS traps the process.

7 Advantages of Segmentation

1. Independent Growth

Each segment can grow or shrink without colliding with unrelated structures.

Example

If the symbol table grows unusually large, it expands inside its own segment instead of overwriting the source-text table.

2. Easier Linking

If each procedure is placed in its own segment, a call to a procedure can use:

$$(\text{segment } n, 0)$$

The entry point remains offset 0 inside that segment. If the procedure is recompiled and becomes larger, other procedures do not need new starting addresses.

3. Easier Sharing

Segments can be shared between processes. For example, a graphical library can be placed in one segment and mapped into many processes.

4. Better Protection

Different segments can have different permissions.

Segment type	Possible protection
Procedure code	Execute only or read/execute.
Constant table	Read only.
Array or heap data	Read/write, but not execute.
Stack	Read/write, but not execute.

This helps catch bugs such as jumping into data or writing into code.

8 Paging vs Segmentation

Feature	Paging	Segmentation
Programmer view	Usually invisible.	Visible logical units.
Address form	One-dimensional virtual address.	Two-part address: segment and offset.
Unit size	Fixed-size pages.	Variable-size segments.
Main purpose	Efficient physical memory management.	Logical organization, growth, sharing, and protection.
Protection	Usually page-level.	Natural per logical object.

9 Final Summary

One-Box Summary

Segmentation gives a process multiple logical address spaces.
A segmented address has the form (s, d) .
Each segment starts at offset 0 and can grow independently.
Segmentation helps with growth, linking, sharing, and protection.